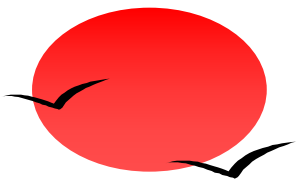


软件设计师

案例分析专题 试题5

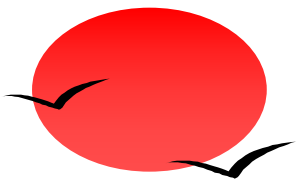
面向对象程序设计

讲师：诸葛老师



1 考点分析

- 根据考试大纲，面向对象程序设计已经成为案例分析的固定**试题五或试题六**，占**15分**，主要考查C++或Java的**基本语法、设计模式**等。
- **1. 面向对象开发**
主要考查面向对象开发中**类、对象、属性、方法、封装、继承、多态、对象之间的引用、接口**等。
- **2. 面向对象设计**
主要考查面向对象设计中体系结构、类的设计、用户接口设计等。
- **3. 设计模式**
主要考查设计模式的**应用场景及其实现**（创建型设计模式、结构型设计模式、行为型设计模式）。



2 面向对象程序设计

■ 1. C++基本语法

类、纯虚函数、派生类、构造函数、析构函数、对象指针和引用。

●类

class 类名

{

public:

公有数据成员或公有函数成员的定义;

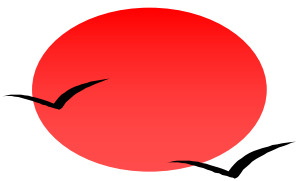
protected:

保护数据成员或保护函数成员的定义;

private:

私有数据成员或私有函数成员的定义;

}



2 面向对象程序设计

●纯虚函数

纯虚函数是一种特殊的虚函数，它的一般格式如下：

```
class <类名>
```

```
{
```

```
    virtual <类型><函数名>(<参数表>)=0;
```

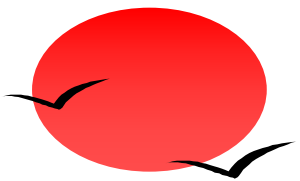
```
    ...
```

```
};
```

虚函数后面加 = 0，表示这个函数为纯虚函数；

纯虚函数的特点：

1. 纯虚函数没有函数体；
2. 一个类里如果包含 = 0 的纯虚函数，那么这个类就是一个抽象类；
3. 抽象类不能具体实例化（不能创建它的对象），而只能由它去派生子类；
4. 在派生类中对此函数提供定义后，它才能具备函数的功能，可被调用。



2 面向对象程序设计

- 派生类

class 派生类名:继承方式1 基类名1,继承方式2 基类名2,...

{

public:

派生类的公有数据和函数;

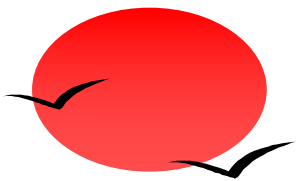
protected:

派生类的保护数据或函数;

private:

派生类的私有数据或函数;

}



2 面向对象程序设计

●构造函数

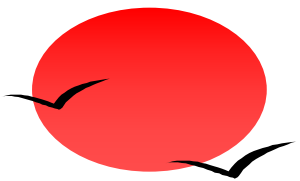
构造函数相对于一般函数来说，具有如下特殊性质：

- (1) 函数名必须与类名一致；
- (2) 没有返回值，在构造函数前加void是错误的；
- (3) 被声明定义为公有函数；
- (4) 在建立对象时由系统自动调用。

●析构函数

析构函数相对于一般函数来说，具有如下性质：

- (1) 没有任何参数，不能被重载，但可以是虚函数，一个类只有一个析构函数；
- (2) 没有返回值，析构函数名与类名相同，但在函数名前加上一个逻辑非运算符“~”，以区分构造函数；
- (3) 一般由用户自己定义，在对象销毁时由系统自动调用，如果用户没有定义析构函数，系统将自动生成一个不做任何事的默认析构函数。



2 面向对象程序设计

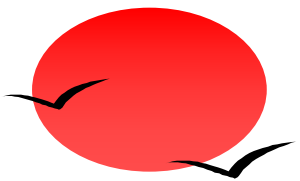
- 对象指针

对象指针的语法定义形式：类名 *对象指针名。

- 对象引用

对象引用的定义形式：类名 &对象引用名 = 被引用对象。

注意：通过对象名或对象引用访问对象的成员，使用的运算符是“.”；而使用对象指针访问对象的成员，使用的运算符是“->”。如：对象指针名->数据成员名或对象指针名->成员函数名(参数表)。



2 面向对象程序设计

■ 2. Java基本语法

类的定义格式如下：

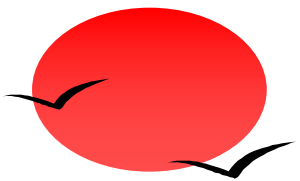
```
[import 包]
[类修饰符] class 类名 [extends 超类] [implements 接口] {
    public/protected/private 数据成员或函数;
}
```

说明：

- import包：引入包中的类。
- 类修饰符：主要有四个修饰符，public、abstract、final、private。
- class为关键字，extends为继承关键字，implements为接口关键字。
- 接口：Java中的接口是一种特殊的类，其内部的实现方法只有定义，没有实现，定义如下：

```
interface IFactory {}
```

```
class SqlServerFactory implements IFactory
```

3 解题技巧

- 第5、6题考试二选一作答，都是程序填空题，原理一模一样，只不过一个用C++语言编写，一个用Java语言编写，并且程序填空只考**基本语法**，基本上不涉及算法，比第4题C语言算法填空简单很多，完全可以拿满分。

如果是初学者，或者对于两种语言都不太精通，**就专攻Java程序题**，因为Java语法比C++要简单并且容易理解记忆，容易拿到满分。

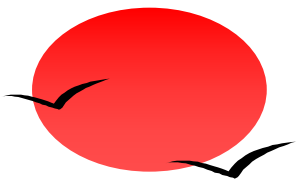
面向对象的程序填空分为两类：**一类是考查纯定义，如接口类，抽象类中的函数定义等；一类是考查函数调用和参数传值**。根据程序代码都可以快速判断出来。

- 注意以下几点：

(1) 定义了类的对象后，必须先初始化（使用new关键字）；

(2) **实现了一个接口，就要实现这个接口中的所有方法；**

(3) 抽象类中可以有普通的方法（有函数体），也可以有抽象方法（无函数体，方法前要加关键字），而接口中的方法都是默认为抽象方法（因为默认，无需再加任何关键字标识）；



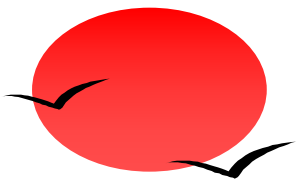
3 解题技巧

(4) **this**的使用，指代当前对象，一般有两个重名变量的赋值时会使用到，如在构造函数中，参数名和私有变量名相同都为name，就要使用`this.name=name`;

(5) 题目所给的类图很重要，从中可以查看类之间的关系以及类中的方法。

(6) 注意实例化类的对象名，通常是类名的小写或缩写。如`Fruit fruit = new Fruit()`或`Fruit f = new Fruit()`;

答案几乎都在题干代码中!



本节练习-面向对象程序设计

- 阅读下列说明和Java代码，将应填入(n)处的字句写在答题纸的对应栏内。

【说明】

Facade(外观)模式是一种通过为多个复杂子系统提供一个一致的接口，而使这些子系统更加容易被访问的模式。以医院为例，就医时患者需要与医院不同的职能部门交互，完成挂号、门诊、取药等操作。为简化就医流程，设置了一个接待员的职位，代患者完成上述就医步骤，患者则只需与接待员交互即可。如图5-1给出了以外观模式实现该场景的类图。

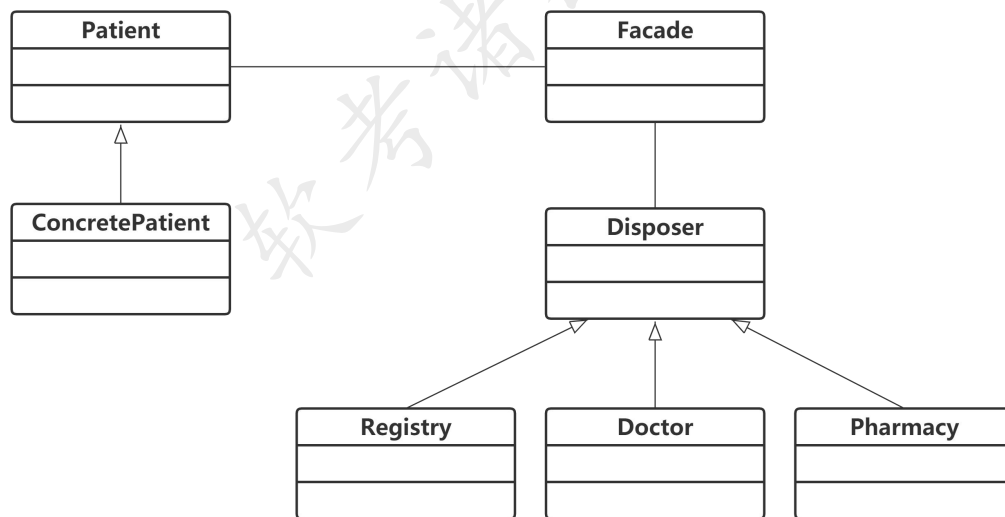
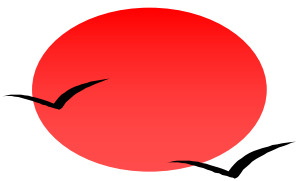


图5-1 类图



本节练习-面向对象程序设计

【Java 代码】

```
import java.util.*;
```

```
Interface Patient{  
    ____ (1) ____;
```

```
}
```

```
Interface Disposer{  
    ____ (2) ____;
```

```
}
```

```
class Registry implements Disposer{//挂号
```

```
    public void dispose(Patient patient){
```

```
        System.out.println("I am registering...."+patient.getName());
```

```
    }
```

```
}
```

```
class Docter implements Disposer{//医生门诊
```

```
    public void dispose(Patient patient){
```

```
        System.out.println("I am diagnosing...."+patient.getName());
```

```
    }
```

```
}
```

```
class Pharmacy implements Disposer{//取药
```

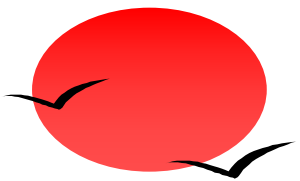
```
    public void dispose(Patient patient){
```

```
        System.out.println("I am giving medicine.”
```

```
+patient.getName());
```

```
    }
```

```
}
```



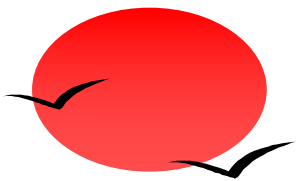
本节练习-面向对象程序设计

```
class Facade{
    private Patient patient;
    public Facade(Patient patient){
        this.patient = patient;
    }
    public void dispose(){
        Registry registry = new Registry();
        Doctor doctor = new Doctor();
        Pharmacy ph = new Pharmacy();

        registry.dispose(patient);
        doctor.dispose(patient);
        ph.dispose(patient);
    }
}
```

```
class ConcretePatient implements Patient{
    private String name;
    public ConcretePatient(String name){
        this.name = name;
    }
    public String getName(){
        return name;
    }
}

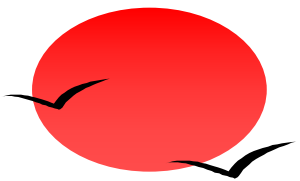
class FacadeTest{
    public static void main(String[] args){
        Patient patient = __ (3) __;
        __ (4) __ f = __ (5) __;
        __ (6) __;
    }
}
```



本节练习-面向对象程序设计

■ 【参考答案】

- (1) `public String getName()`
- (2) `public void dispose(Patient patient)`
- (3) `new ConcretePatient("zhangsan")`
- (4) `Facade`
- (5) `new Facade(patient)`
- (6) `f.dispose()`



本节练习-面向对象程序设计

- 阅读下列说明和C++代码，将应填入 (n) 处的字句写在答题纸的对应栏内。

【说明】

Facade(外观)模式是一种通过为多个复杂子系统提供一个一致的接口，而使这些子系统更加容易被访问的模式。以医院为例，就医时患者需要与医院不同的职能部门交互，完成挂号、门诊、取药等操作。为简化就医流程，设置了一个接待员的职位，代患者完成上述就医步骤，患者则只需与接待员交互即可。如图5-1给出了以外观模式实现该场景的类图。

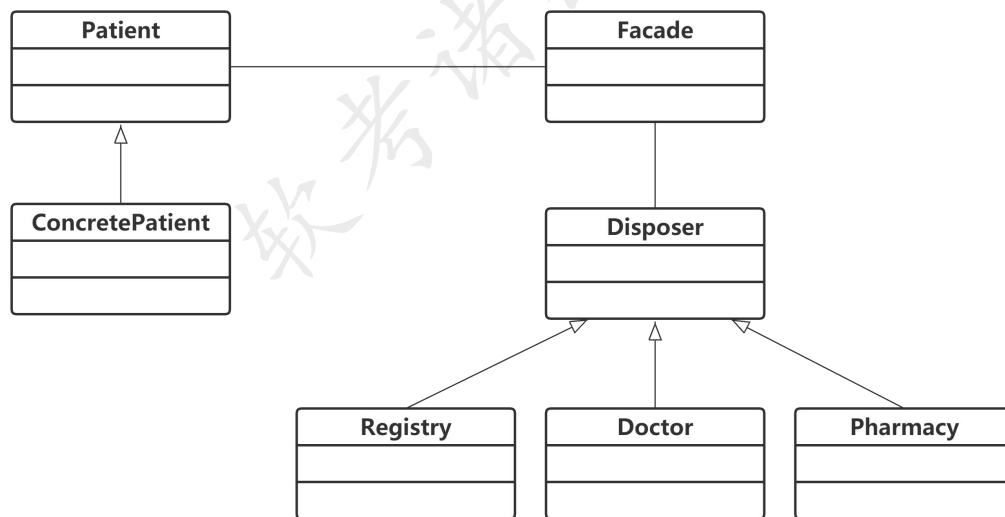
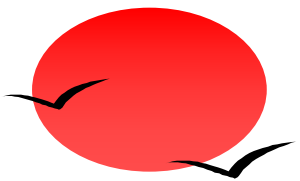


图5-1 类图



本节练习-面向对象程序设计

【C++ 代码】

```
#include<iostream>
#include<string>
using namespace std;
```

```
class Patient{
public:
    ____ (1) ____;
```

```
class Disposer{
public:
    ____ (2) ____;
}
```

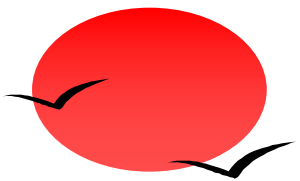
```
class Registry:public Disposer{//挂号
public:
    void dispose(Patient *patient){
        cout<<“I am registering....”<<patient->getName()<<endl;
    }
}
```

```
class Doctor:public Disposer{//医生门诊
public:
```

```
    void dispose(Patient *patient){
        cout<<“I am diagnosing....”<<patient->getName()<<endl;
    }
}
```

```
class Pharmacy:public Disposer{//取药
public:
```

```
    void dispose(Patient *patient){
        cout<<“I am giving medicine...”<<patient->getName()<<endl;
    }
}
```

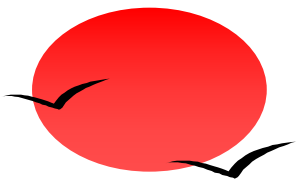
本节练习-面向对象程序设计

```
class Facade{
private:
    Patient *patient;
public:
    Facade(Patient *patient){
        this->patient = patient;
    }
    void dispose(){
        Registry *registry = new Registry();
        Doctor *doctor = new Doctor();
        Pharmacy *ph = new Pharmacy();

        registry->dispose(patient);
        doctor->dispose(patient);
        ph->dispose(patient);
    }
}
```

```
class ConcretePatient: public Patient{
private:
    string name;
public:
    ConcretePatient(String name){
        this->name = name;
    }
    String getName(){return name;}
}

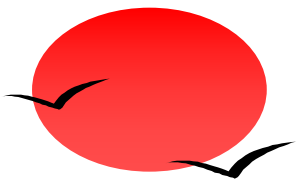
int main{
    Patient *patient = __ (3) __;
    __ (4) __ f = __ (5) __;
    __ (6) __;
    return 0;
}
```



本节练习-面向对象程序设计

■ 【参考答案】

- (1) virtual string getName() = 0
- (2) virtual void dispose(Patient *patient) = 0
- (3) new ConcretePatient("zhangsan")
- (4) Facade*
- (5) new Facade(patient)
- (6) f->dispose()



本节练习-面向对象程序设计

- 阅读下列说明和Java代码，将应填入(n)处的字句写在答题纸的对应栏内。

【说明】

在软件系统中，通常都会给用户提供取消、不确定或者错误操作的选择，允许将系统恢复到原先的状态。现使用备忘录（Memento）模式实现该要求，得到如图6-1所示的类图。Memento包含了要被恢复的状态。Originator创建并在Memento中存储状态。Caretaker负责从Memento中恢复状态。

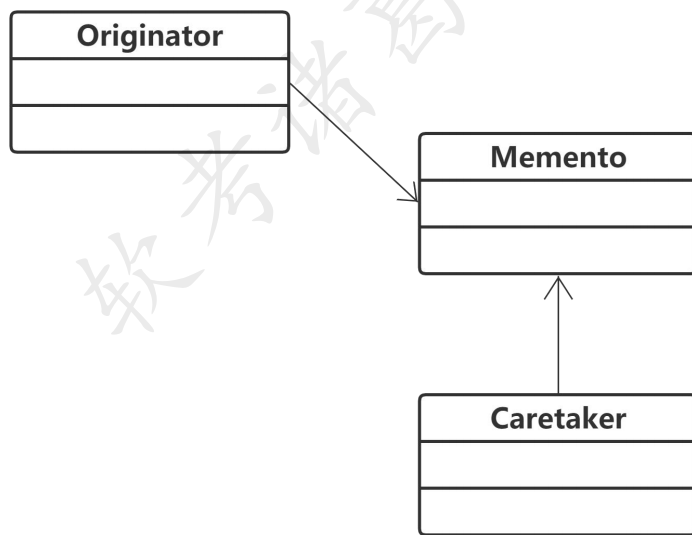
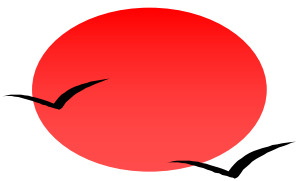


图5-1 类图



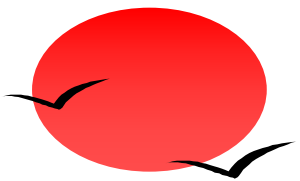
本节练习-面向对象程序设计

【Java 代码】

```
import java.util.*;
```

```
class Memento{  
    private String state;  
    public memento(String state){  
        this.state = state;  
    }  
    public String getState(){  
        return state;  
    }  
}
```

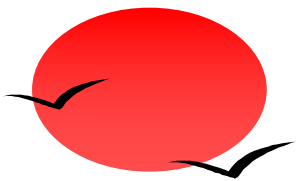
```
class Originator{  
    private String state;  
    public void setState(String state){  
        this.state = state;  
    }  
    public String getState(){ return state;}  
    public Memento saveStateToMemento(){  
        return __ (1) __;  
    }  
    public void getStateFromMemento(Memento memento){  
        state = __ (2) __;  
    }  
}
```



本节练习-面向对象程序设计

```
class CareTaker{  
    private List<Memento> mementoList  
        = new ArrayList<Memento>();  
    public __ (3) __ {  
        mementoList.add(state);  
    }  
    public __ (4) __ {  
        return mementoList.get(index);  
    }  
}
```

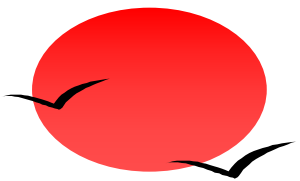
```
class MementoPaneDemos{  
    public static void main(String[] args){  
        Originator originator = new Originator();  
        CareTaker careTaker = new CareTaker();  
        originator.setState("State #1");  
        originator.setState("State #2");  
        careTaker.add(__ (5) __);  
        originator.setState("State #3");  
        careTaker.add(__ (6) __);  
        originator.setState("State #4");  
        System.out.println("Current State:" + originator.getState());  
        originator.getStateFromMemento(careTaker.get(0));  
        System.out.println("First Saved State:" + originator.getState());  
        originator.getStateFromMemento(careTaker.get(1));  
        System.out.println("Second Saved State:" + originator.getState());  
    }  
}
```



本节练习-面向对象程序设计

■ 【参考答案】

- (1) new Memento(state)
- (2) Memento.getState()
- (3) void add(Memento state)
- (4) Memento get(int index)
- (5) originator.saveStateToMemento()
- (6) originator.saveStateToMemento()



本节练习-面向对象程序设计

- 阅读下列说明和C++代码，将应填入(n)处的字句写在答题纸的对应栏内。

【说明】

在软件系统中，通常都会给用户提供取消、不确定或者错误操作的选择，允许将系统恢复到原先的状态。现使用备忘录（Memento）模式实现该要求，得到如图6-1所示的类图。Memento包含了要被恢复的状态。Originator创建并在Memento中存储状态。Caretaker负责从Memento中恢复状态。

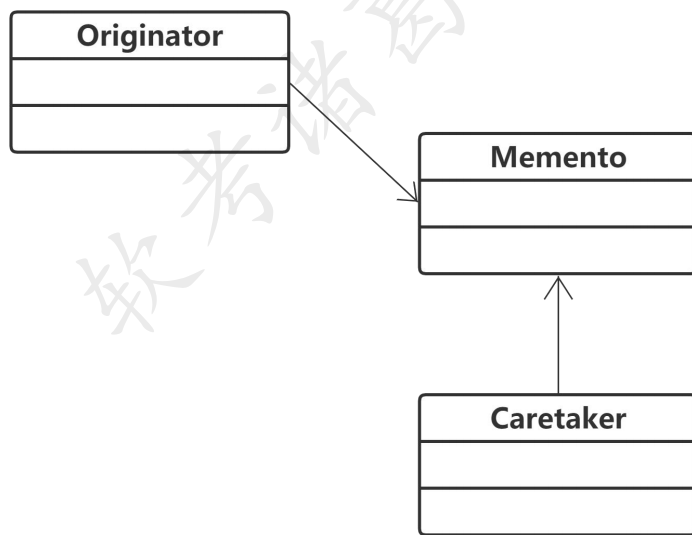
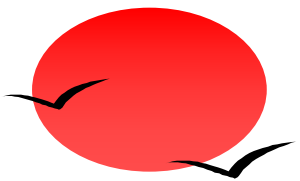


图5-1 类图



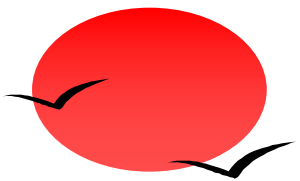
本节练习-面向对象程序设计

【C++ 代码】

```
#include<iostream>
#include<string>
#include<vector>
using namespace std;
```

```
class Memento{
private:
    String state;
public:
    Memento(String state){
        this->state = state;
    }
    String getState(){
        return state;
    }
}
```

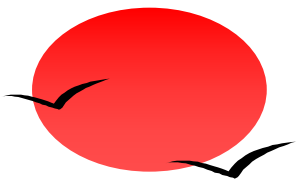
```
class Originator{
private:
    String state;
public:
    void setState(String state){
        this.state = state;
    }
    String getState(){
        return state;
    }
    Memento saveStateToMemento(){
        return __ (1) __;
    }
    void getStateFromMemento(Memento *memento){
        state = __ (2) __;
    }
}
```

本节练习-面向对象程序设计

```
class CareTaker{
private:
    vector<Memento> mementoList;
public:
    (3) {
        mementoList.push_back(state);
    }
    (4) {
        return mementoList.get(index);
    }
}
```

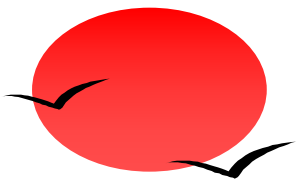
```
int main{
    Originator *originator = new Originator();
    CareTaker *careTaker = new CareTaker();
    originator->setState("State #1");
    originator->setState("State #2");
    careTaker->add( (5) );
    originator->setState("State #3");
    careTaker->add( (6) );
    originator->setState("State #4");
    cout<<"Current State:"<<"+"<<originator->getState()<<endl;
    originator->getStateFromMemento(careTaker->get(0));
    cout<<"First Saved State:"<<"+"<<originator->getState()<<endl;
    originator->getStateFromMemento(careTaker->get(1));
    cout<<"Second Saved State:"<<"+"<<originator->getState()<<endl;
    return 0;
}
```



本节练习-面向对象程序设计

■ 【参考答案】

- (1) new Memento(state)
- (2) memento->getState()
- (3) add(Memento *state)
- (4) Memento* get(int index)
- (5) originator->saveStateToMemento()
- (6) originator->saveStateToMemento()



感谢

THANKS

软件设计师QQ群: 759713504

诸葛老师微信: zhugeruankao